

CHAPTER -4

ANALYZING FUNCTIONAL DEPENDENCIES, REDUNDANCY AND DATA ANOMALIES

4.1 Introduction

The **Snapdeal E-Commerce Database** consists of multiple related tables that store information about customers, sellers, products, categories, orders, payments, deliveries, and reviews. To ensure that the database is efficient and reliable, it is important to analyze the **functional dependencies**, **data redundancy**, and **data anomalies** present in each table.

A **functional dependency** defines the relationship between the primary key and other attributes in a table. It ensures that every record is uniquely identified and that all non-key attributes depend only on the primary key. **Redundancy analysis** helps identify and eliminate unnecessary duplication of data, thereby improving storage efficiency and maintaining consistency. **Data anomaly analysis** examines insertion, update, and deletion issues that may occur if the database is not properly designed.

This chapter evaluates each entity in the Snapdeal E-Commerce Database to verify that the database structure follows normalization principles and supports accurate, consistent, and efficient data management.

4.2 Customer Table Analysis

Functional Dependency

Customer_ID → **Customer_Name, Email, Phone_Number, Address, Password**

The **Customer_ID** uniquely identifies every customer and determines all other customer attributes.

Redundancy Analysis

Customer information is stored only once in the Customer table. Unique constraints on **Email** and **Phone_Number** prevent duplicate customer records and minimize data redundancy.

Data Anomalies

Insertion Anomaly: A customer record must be created before placing an order.

Update Anomaly: Any change to customer details, such as address or phone number, is updated in a single record.

Deletion Anomaly: Deleting a customer record removes all customer information from the database.

4.3 Seller Table Analysis

Functional Dependency

Seller_ID → **Seller_Name, Email, Phone_Number, Address**

Seller_ID uniquely determines all seller information.

Redundancy Analysis

Seller information is stored only once, and duplicate records are prevented through unique email and phone number constraints.

Data Anomalies

Insertion Anomaly: A seller must be registered before products can be listed.

Update Anomaly: Seller details are updated in one place without affecting other records.

Deletion Anomaly: Deleting a seller removes seller information while maintaining database consistency.

4.4 Category Table Analysis

Functional Dependency

Category_ID → **Category_Name, Description**

Category_ID uniquely identifies every product category.

Redundancy Analysis

Each category is stored only once, preventing repeated category names and descriptions.

Data Anomalies

Insertion Anomaly: Categories must be created before assigning products.

Update Anomaly: Category information is modified in only one record.

Deletion Anomaly: Removing a category deletes its associated category information.

4.5 Product Table Analysis

Functional Dependency

Product_ID → Product_Name, Category_ID, Seller_ID, Price, Stock, Description

Product_ID uniquely identifies every product.

Redundancy Analysis

Category and seller details are referenced using foreign keys instead of storing duplicate information, thereby reducing redundancy.

Data Anomalies

Insertion Anomaly: A product cannot be added unless a valid seller and category already exist.

Update Anomaly: Product information such as price or stock is updated in one location only.

Deletion Anomaly: Removing a product does not affect seller or category records.

4.6 Orders Table Analysis

Functional Dependency

Order_ID → Customer_ID, Order_Date, Total_Amount, Order_Status

Each Order_ID uniquely identifies an order and its details.

Redundancy Analysis

Customer information is not duplicated because the Orders table stores only Customer_ID as a foreign key.

Data Anomalies

Insertion Anomaly: Orders cannot be created without an existing customer.

Update Anomaly: Order status can be updated independently.

Deletion Anomaly: Deleting an order does not remove customer information.

4.7 Payment Table Analysis

Functional Dependency

Payment_ID → **Order_ID**, **Payment_Method**, **Payment_Status**, **Payment_Date**, **Amount**

Payment_ID uniquely determines all payment-related information.

Redundancy Analysis

Payment information is stored separately and linked to Orders through Order_ID, avoiding duplicate payment records.

Data Anomalies

Insertion Anomaly: A payment record cannot exist without a corresponding order.

Update Anomaly: Payment status can be modified without affecting other tables.

Deletion Anomaly: Removing payment information does not delete the associated order.

4.8 Delivery Table Analysis

Functional Dependency

Delivery_ID → **Order_ID**, **Delivery_Address**, **Delivery_Date**, **Delivery_Status**

Delivery_ID uniquely identifies every delivery record.

Redundancy Analysis

Delivery information is stored independently and linked to Orders through Order_ID, eliminating repeated delivery details.

Data Anomalies

Insertion Anomaly: Delivery details can be entered only after an order is placed.

Update Anomaly: Delivery status is updated in one record only.

Deletion Anomaly: Removing delivery information does not affect order details.

4.9 Review Table Analysis

Functional Dependency

Review_ID → **Customer_ID, Product_ID, Rating, Review_Text, Review_Date**

Review_ID uniquely identifies every review submitted by a customer.

Redundancy Analysis

Customer and product information are referenced using foreign keys instead of being duplicated, thereby maintaining consistency.

Data Anomalies

Insertion Anomaly: Reviews can be added only after a customer interacts with a product.

Update Anomaly: Ratings or review comments can be updated independently.

Deletion Anomaly: Deleting a review does not affect customer or product information.

4.10 Conclusion

The analysis of functional dependencies, redundancy, and data anomalies demonstrates that the **Snapdeal E-Commerce Database** is well structured and follows the principles of relational database design. Each table has a clearly defined **Primary Key**, while **Foreign Keys** establish relationships between related entities. This structure ensures that every non-key attribute depends only on the primary key, thereby satisfying normalization principles.

The database design effectively minimizes data redundancy by storing information in separate tables and linking them through relationships. It also prevents insertion, update, and deletion anomalies, ensuring data consistency and integrity throughout the system. Overall, the database provides a scalable, efficient, and reliable foundation for managing customers, sellers, products, orders, payments, deliveries, and reviews in an e-commerce environment.